

SKILL: taller-bv-dashboard-tech

Código exacto para extraer, aplicar y validar cambios en el Dashboard Taller BV.

Rutas

- Dashboard (bash VM): /sessions/*/mnt/Taller BV/Dashboard_Pedidos.html
- XLSX descargado: /tmp/urgentes.xlsx

Decodificar XLSX desde base64

```
import json, base64
d=json.load(open('<ruta_del_txt>'))
raw=base64.b64decode(d['content'])
open('/tmp/urgentes.xlsx','wb').write(raw) # raw[:2]==b'PK'
```

Estructura de los bloques de mes (IMPORTANTE)

En cada hoja OF-* los meses van uno al lado del otro (Mayo, Junio, Julio, Agosto...).

Para cada mes, en la fila 1:

- Los objetivos semanales obj. S.1...obj. S.5 van **ANTES** del label mensual (ocupan las columnas de las fechas de ese mes). El valor está en label+1.
- El label objetivo <Mes> (valor mensual en col+1) va **al final** del bloque.

Para leer los semanales hay que mirar el rango **entre el objetivo del mes anterior y el objetivo de este mes** — NO escanear hacia adelante (agarraría los del mes siguiente).

Semana de 1 día sin rótulo: si una semana (S1 o S5) tiene un solo día hábil, en el sheet NO lleva el rótulo obj. S.x — el objetivo queda como valor suelto en fila 1 de esa columna. Se lee **posicionalmente** (1er valor numérico de fila 1 de la semana que no venga después de un Prod. S.) y se **valida** con Σ semanales == objetivo mensual (si no cuadra y hay una sola semana sin rótulo, se corrige con el residual mensual - Σ rotuladas).

Extraer producción por GA (openpyxl) — DETECCIÓN AUTOMÁTICA DE MES

Regla de meses: detectar por el rótulo objetivo <Mes> + columnas de fecha. Sincronizar el mes en curso (AR) + meses futuros ya armados. NUNCA re-sincronizar meses históricos congelados. Mes con columnas pero sin objetivo → bloque vacío (obj 0, semanas [], codigos []).

Estatores: ignorar la columna 30/04. No contar filas con código "-". Semanales al entero.

```
import openpyxl
from datetime import datetime, timezone, timedelta
from collections import defaultdict

wb = openpyxl.load_workbook('/tmp/urgentes.xlsx', data_only=True)
MESES_ES = {1:'Enero',2:'Febrero',3:'Marzo',4:'Abril',5:'Mayo',6:'Junio',
            7:'Julio',8:'Agosto',9:'Septiembre',10:'Octubre',11:'Noviembre',12:'Diciembre'}

def extraer_ga(sheet_name, year, month):
    ws = wb[sheet_name]; mes_nombre = MESES_ES[month]; maxc = ws.max_column
    row1 = {c: ws.cell(1, c).value for c in range(1, maxc+1)}

    lab_c = None
    for c in range(1, maxc+1):
        v = row1[c]
        if isinstance(v, str) and v.strip().rstrip(':').lower() == ('objetivo '+mes_nombre).lower():
            lab_c = c; break
    obj_mensual = row1.get(lab_c+1) if lab_c else None

    # semanales rotulados: entre el 'Objetivo' anterior y este (NO hacia adelante)
```

```

obj_sems = {}
if lab_c:
    prev = 0
    for c in range(lab_c-1, 0, -1):
        v = row1.get(c)
        if isinstance(v, str) and v.strip().lower().startswith('objetivo'):
            prev = c; break
    for c in range(prev+1, lab_c):
        v = row1.get(c)
        if isinstance(v, str) and 'Obj. S.' in v:
            try:
                num = int(v.split('.')[0].strip()); val = row1.get(c+1)
                if isinstance(val, (int, float)): obj_sems[num] = round(val)
            except: pass

# columnas de fecha del mes/año (Estatores: ignorar 30/04)
cols = []
for c in range(1, maxc+1):
    v = ws.cell(2, c).value
    if isinstance(v, datetime) and v.year == year and v.month == month:
        if sheet_name == 'OF-EST' and v.month == 4 and v.day == 30: continue
        cols.append((c, v))
if not cols:
    return None

# producción total (excluir filas con código '-'/vacío)
total = 0; codigos = []
for row in range(3, ws.max_row+1):
    cod = ws.cell(row, 1).value
    if not cod or not isinstance(cod, str): continue
    cod = cod.strip()
    if cod in ('-', ''): continue
    p = sum(ws.cell(row,c).value or 0 for c,_ in cols if isinstance(ws.cell(row,c).value,(int,float)))
    if p > 0:
        cat = ws.cell(row, 2).value; total += p
        codigos.append({'codigo': cod.replace(',','.'),
                        'categoria': str(cat).strip() if cat else '', 'cantidad': int(p)})

obj_m = round(obj_mensual) if isinstance(obj_mensual, (int, float)) else 0
if obj_m == 0 and total == 0:
    return {'mes_key': f'{year:04d}-{month:02d}', 'mes_label': f'{mes_nombre} {year}',
            'obj_mensual': 0, 'dias_habiles': len(cols),
            'prod_total': 0, 'semanas': [], 'codigos': []}

# semanas (por número ISO). Semana sin rótulo -> lectura posicional.
sem_map = defaultdict(list)
for c, dt in sorted(cols, key=lambda x: x[1]):
    sem_map[dt.isocalendar()[1]].append((c, dt))
semanas = []
for i, (wk, ccols) in enumerate(sorted(sem_map.items())):
    wn = i+1
    objw = obj_sems.get(wn)
    if objw is None:
        for c, _ in ccols:
            v = row1.get(c); pv = row1.get(c-1)
            if isinstance(v, (int, float)) and not (isinstance(pv, str) and 'Prod' in pv):
                objw = round(v); break
        if objw is None: objw = 0
    prod_s = 0
    for row in range(3, ws.max_row+1):
        cod = ws.cell(row,1).value
        if not cod or not isinstance(cod,str) or cod.strip() in ('-', ''): continue
        prod_s += sum(ws.cell(row,c).value or 0 for c,_ in ccols if isinstance(ws.cell(row,c).value,(int,float)))
    semanas.append({'num': wn, 'dias': len(ccols), 'objw': objw, 'prod': int(prod_s)})

# Validación: Σ semanales == mensual. Corregir semana sin rótulo con residual o avisar.
if obj_m:
    _suma = sum(x['objw'] for x in semanas)
    if _suma != obj_m:
        _sin = [x for x in semanas if x['num'] not in obj_sems]
        if len(_sin) == 1:
            _sin[0]['objw'] = obj_m - sum(x['objw'] for x in semanas if x['num'] in obj_sems)
        else:
            print(f"[AVISO] {sheet_name} {mes_nombre}: suma semanales ({_suma}) != mensual ({obj_m}) - revisar she

return {'mes_key': f'{year:04d}-{month:02d}', 'mes_label': f'{mes_nombre} {year}',
        'obj_mensual': obj_m, 'dias_habiles': len(cols),
        'prod_total': int(total), 'semanas': semanas, 'codigos': codigos}

# Meses a sincronizar: mes en curso (AR) + meses futuros ya armados
hoy = datetime.now(timezone(timedelta(hours=-3)))
objetivo_meses = [(hoy.year, hoy.month)]
for m in range(hoy.month + 1, 13):

```

```

        if extraer_ga('OF-IND', hoy.year, m) is not None:
            objetivo_meses.append((hoy.year, m))
    bloques = {}
    for (Y, M) in objetivo_meses:
        for ga, sh in [('Inducidos', 'OF-IND'), ('Rotores', 'OF-ROT'), ('Estatores', 'OF-EST')]:
            r = extraer_ga(sh, Y, M)
            if r: bloques.setdefault(r['mes_key'], {})[ga] = r

```

Aplicar al HTML: para cada mes_key/ga en bloques, actualizar la entrada de OF_DATA

(obj_mensual, prod_total, semanas, codigos, dias_habiles) o agregarla si es mes nuevo.

Ordenar OF_DATA por mes_key y GA (Inducidos, Rotores, Estatores). No tocar meses fuera de bloques.

Extraer pedidos (hoja Urgentes)

Col 1=N, 2=Ingreso, 3=GA, 4=Cod, 5=Cond, 6=Cliente, 7=Días obj, 8=Entrega sol, 12=Reclamo,

13=Obs, 14=Entrega real, 16=Días prod (P), 17=Demora (Q).

```

def fmt_date(v):
    if isinstance(v, datetime): return v.strftime('%Y-%m-%d')
    if isinstance(v, str) and v not in ('-', ''): return v
    return None
ws = wb['Urgentes']; pedidos_sheet = {}
for row in range(3, ws.max_row+1):
    n = ws.cell(row,1).value; ga = ws.cell(row,3).value
    if not isinstance(n,(int,float)) or not isinstance(ga,str): continue
    n = int(n); dpr = ws.cell(row,16).value
    dias_prod = int(dpr) if isinstance(dpr,(int,float)) and dpr >= 0 else None # negativo -> null
    cod = ws.cell(row,4).value; cod = str(cod).strip().replace(',','.') if cod else None
    pedidos_sheet[n] = {'n': n, 'ga': str(ga).strip(), 'codigo': cod,
        'condicion': ws.cell(row,5).value, 'cliente': ws.cell(row,6).value,
        'fecha_ingreso': fmt_date(ws.cell(row,2).value),
        'dias_objetivo': int(ws.cell(row,7).value) if isinstance(ws.cell(row,7).value,(int,float)) else None,
        'entrega_sol': fmt_date(ws.cell(row,8).value),
        'reclamo': ws.cell(row,12).value if ws.cell(row,12).value not in (None,'-') else None,
        'observacion': ws.cell(row,13).value if ws.cell(row,13).value not in (None,'-') else None,
        'fecha_entrega_real': fmt_date(ws.cell(row,14).value),
        'dias_prod': dias_prod,
        'demora': round(float(ws.cell(row,17).value),4) if isinstance(ws.cell(row,17).value,(int,float)) else None}

```

Actualizar badge

```

import re
from datetime import datetime, timezone, timedelta
AR = timezone(timedelta(hours=-3)); ahora = datetime.now(AR)
dias = ['Lun','Mar','Mié','Jue','Vie','Sáb','Dom']
fecha = f"{dias[ahora.weekday()]} {ahora.strftime('%d/%m/%Y %H:%M')}"
html = re.sub(r'const _ULTIMO_SYNC='[^']*"', f'const _ULTIMO_SYNC="{fecha}"', html) # NO html.find

```

No sobrescribir con vacíos

```

def es_cambio(campo, v_dash, v_sheet):
    if v_sheet is None: return False
    if v_dash == v_sheet: return False
    if campo == 'demora' and v_dash is not None and round(float(v_dash),2)==round(float(v_sheet),2): return False
    if campo == 'cliente' and str(v_dash or '').lower()==str(v_sheet or '').lower(): return False
    if campo == 'observacion' and str(v_dash or '').strip()==str(v_sheet or '').strip(): return False
    return True

```

Serializador OF_DATA / PEDIDOS (JS-literal)

Claves sin comillas, strings con comilla simple, null para vacíos, cerrar con] ; .

Validación post-aplicación (Node.js)

Parsear OF_DATA y PEDIDOS con new Function(...); deben pasar sin error. Chequear

mes_keys, longitudes, y que los objetivos semanales del mes en curso NO sean todos 0

si el mes tiene objetivo (y que Σ semanales == objetivo mensual).

Criterio de "% prom." en Urgentes (23/07/2026)

Tarjeta General y solapas por GA: % prom = (días prom – OBJ_SERV) / OBJ_SERV. Selector OBJ_SERV (3/2/1, default 3, simulación). Verde adelanto, rojo retraso.

Días prom. / % prom.: solo entregados (29/07/2026)

Días prod. prom., % prom. y avgDemDP se calculan SOLO sobre pedidos ENTREGADOS (fecha_entrega_real); en proceso no cuentan. Helpers: entregadoP(p), dpProm(arr).

Selector de "días de servicio" (simulación) y % Servicio (04/08/2026)

OBJ_SERV (3/2/1) también recalcula el % **Servicio**: objetivo efectivo de cada pedido = $G - (3 - \text{OBJ_SERV})$ (baja el base 3, mantiene el extra de los especiales), recomputando la fecha comprometida = ingreso + objetivo efectivo (días hábiles). En demoraEfectiva: si OBJ_SERV===3 usa los valores del sheet; si no, recalcula con _bdays/_addBdays. El detalle "Ver cálculo" ordena por GA → alfabético → especiales al final, y recalcula Días y % Servicio con el selector. El encabezado (header + toolbar) es sticky (.topbar).